



Lecture 9. Input and output

Functional Programming

- This course: typed, *purely* functional programming
- Today: how can a *pure* program interact with the outside world?

Goals

- Learn the difference between pure and impure
- Interact with the outside world in Haskell
 - Input/output
 - Random generation
- Introduce do- and monadic notation through an example

Chapter 10 from Hutton's book

Interactive programs

- In the old days, many programs were *batch* programs
 - Introduce the program and input, sit and drink tea/coffee for hours, and get the output
 - Programs were isolated from each other
 - The part of Haskell you have learnt up to now

Interactive programs

- In the old days, many programs were *batch* programs
 - Introduce the program and input, sit and drink tea/coffee for hours, and get the output
 - Programs were isolated from each other
 - The part of Haskell you have learnt up to now
- Many modern programs are *interactive*
 - Respond to user input, more like a dialogue
 - From the perspective of a program, it needs to communicate with an *outside world*
 - **Examples?**
 - Today: how we model this in Haskell!

Interactive programs

- In the old days, many programs were *batch* programs
 - Introduce the program and input, sit and drink tea/coffee for hours, and get the output
 - Programs were isolated from each other
 - The part of Haskell you have learnt up to now
- Many modern programs are *interactive*
 - Respond to user input, more like a dialogue
 - From the perspective of a program, it needs to communicate with an *outside world*
 - **Examples?**
 - Today: how we model this in Haskell!
- For your game: keep state updates pure; use IO at the boundary.

Purity = referential transparency

Referential transparency = replacing an expression by an equal expression does not change program meaning

Purity = referential transparency

Referential transparency = replacing an expression by an equal expression does not change program meaning

- *Inlining* (non-recursive binding, at a fixed type):

let $x = e$ **in** ... x ... x ...

is by definition equivalent to:

($\lambda x \rightarrow$... x ... x ...) e

is by definition equivalent to:

... x ... x ... **where** $x = e$

is (because we may inline) equivalent to:

... e ... e ...

Referential transparency

A concrete example:

```
reverse xs ++ xs  
  where xs = filter p ys
```

is equivalent to:

```
reverse (filter p ys) ++ filter p ys
```

Referential transparency

A concrete example:

```
reverse xs ++ xs  
  where xs = filter p ys
```

is equivalent to:

```
reverse (filter p ys) ++ filter p ys
```

Note that the second version duplicates work, but we are speaking here about the *meaning* of the expression, not its efficiency

Referential transparency: why care?

Assume non-recursive bindings; substitutions avoid capture.

Referential transparency: why care?

Assume non-recursive bindings; substitutions avoid capture.

- *Copying/duplication (contraction)*

```
let x1 = e; x2 = e in t
```

```
let x1 = e in t[x1/x2]    -- equivalent
```

Referential transparency: why care?

Assume non-recursive bindings; substitutions avoid capture.

- *Copying/duplication (contraction)*

```
let x1 = e; x2 = e in t
```

```
let x1 = e in t[x1/x2]    -- equivalent
```

- *Discarding (weakening)*, if t does not mention x

```
let x = e in t
```

```
t                        -- equivalent
```

Referential transparency: why care?

Assume non-recursive bindings; substitutions avoid capture.

- *Copying/duplication (contraction)*

```
let x1 = e; x2 = e in t
let x1 = e in t[x1/x2]    -- equivalent
```

- *Discarding (weakening), if t does not mention x*

```
let x = e in t
t                          -- equivalent
```

- *Commuting/reordering (exchange)*

```
let x1 = e1; x2 = e2 in t
let x2 = e2; x1 = e1 in t -- equivalent
```

Referential transparency

- Referential transparency decouples the meaning of the program from the order of evaluation
 - Inlining or duplicating does not change the program

Referential transparency

- Referential transparency decouples the meaning of the program from the order of evaluation
 - Inlining or duplicating does not change the program
- This has practical advantages:
 - The compiler can reorder your program for efficiency
 - Haskell delays evaluation until a value is needed
 - *Lazy evaluation* also shares evaluated results
 - Parallelism becomes much easier

Performing an effect cannot in general be duplicated or reordered freely.

A hypothetical design (not Haskell)

Performing an effect cannot in general be duplicated or reordered freely.

A hypothetical design (not Haskell)

Suppose evaluating `getChar :: Char` reads the next character

Assume this binding reads once:

```
let k = getChar in k == k
```

Would inlining `k` preserve the behaviour? Explain the difference.

Side-effects

Performing an effect cannot in general be duplicated or reordered freely.

A hypothetical design (not Haskell)

Suppose evaluating `getChar :: Char` reads the next character

```
let k = getChar in k == k
```

reads once and returns `True` (if the read succeeds), unlike

```
getChar == getChar
```

Reading input is a **side effect**.

- Real Haskell uses `getChar :: IO Char`, not `Char`.

- Many other actions have side-effects (why?)
 - Printing to the screen
 - Drawing from a shared random generator
 - Communicate through a network
 - Talk to a database
- These actions observe or change the outside world
 - Their execution cannot generally be discarded, duplicated or reordered
 - Their *descriptions* remain ordinary Haskell values

Haskell typing of code with IO/side-effects

```
getChar  :: IO Char
getLine  :: IO String
getArgs  :: IO [String]

putChar  :: Char    -> IO ()
putStr   :: String  -> IO ()
putStrLn :: String  -> IO ()
-- ...
```

Haskell typing of code with IO/side-effects

```
return    :: a -> IO a
(>>=)     :: IO a -> (a -> IO b) -> IO b
-- Sequences actions; passes a result to the next action.
-- IO expressions still obey referential transparency.

getChar   :: IO Char
getLine   :: IO String
getArgs   :: IO [String]

putChar   :: Char -> IO ()
putStr    :: String -> IO ()
putStrLn  :: String -> IO ()
-- ...
```

A model of IO

Modelling output

As a **conceptual model**, represent an action by a function which changes the world

```
type IOCom = World -> World -- IO ()
```

Using IOCom we can give a type to putChar

```
putChar :: Char -> IOCom  
putChar c world = ... -- details hidden
```

Modelling output

As a **conceptual model**, represent an action by a function which changes the world

```
type IOCom = World -> World -- IO ()
```

Using IOCom we can give a type to putChar

```
putChar :: Char -> IOCom  
putChar c world = ... -- details hidden
```

How should we think of World and putChar?

Combining output actions

Executing two actions in sequence is plain composition

```
putAB :: IOCom  
putAB world = putChar 'b' (putChar 'a' world)  
-- or using composition  
putAB      = putChar 'b' . putChar 'a'
```

putStr, first version

`putStr s` prints the whole string to the screen

```
putStr :: String -> IOCom
```

```
putStr []      = id  -- keep the world as it is
```

```
putStr (c:cs) = putStr cs . putChar c
```

`putStrLn s` does the same, with a newline at the end

```
putStrLn s = putChar '\n' . putStr s
```

Our IOCom type is not suitable for `getChar`. Why not? Fix?

Modelling input

Our IOCom type is not suitable for getChar. Why not? Fix?

- Solution: pair the output value with the new world

```
type IO a = World -> (a, World) -- conceptual model only
```

```
getChar :: IO Char
```

```
getChar = ... -- details hidden
```

What is now the return type of putChar?

Modelling input

Our IOCom type is not suitable for getChar. Why not? Fix?

- Solution: pair the output value with the new world

```
type IO a = World -> (a, World) -- conceptual model only
```

```
getChar :: IO Char
```

```
getChar = ... -- details hidden
```

What is now the return type of putChar?

- We use the empty tuple as a dummy value

```
putChar :: Char -> IO ()
```

Combining input and output

Suppose that we want to echo a character

```
echo = putChar getChar
```

- Couldn't match expected type 'Char'
with actual type 'IO Char'

Combining input and output

Let's try again with function composition

```
echo = putChar . getChar  -- also a type error
```

```
getChar :: IO Char
          -- World -> (Char, World)
putChar :: Char -> IO ()
          -- Char -> World -> ((), World)
(.)      :: (b -> c) -> (a -> b) -> a -> c
```

Types do not fit, since `b` should be both `(Char, World)` – from `getChar` – and `Char` – from `putChar`

Solution: bind

(>>=) – pronounced “bind” – takes care of threading the world around

```
(>>=) :: IO a -> (a -> IO b) -> IO b
```

```
(f >>= g) w = ...
```

Based on the output of the first action, we choose which action to perform next

```
echo = getChar >>= \c -> putChar c  
      -- also getChar >>= putChar
```

Solution: bind

(>>=) – pronounced “bind” – takes care of threading the world around

```
(>>=) :: IO a -> (a -> IO b) -> IO b
```

```
(f >>= g) w = g a' w' where
```

```
  (a', w') = f w
```

Based on the output of the first action, we choose which action to perform next

```
echo = getChar >>= \c -> putChar c  
      -- also getChar >>= putChar
```

Uppercase input

We want a `getUpper` action that reads the next character and returns its uppercase version

```
getChar :: IO Char
```

```
toUpper :: Char -> Char
```

```
getUpper = getChar >>= \c -> toUpper c
```

- Couldn't match expected type 'IO Char'
with actual type 'Char'

Uppercase input

We need a way to *embed* pure computations, like `toUpper`, in the impure world

```
return :: a -> IO a
```

```
return a = ...
```

Warning! `return` is indeed a very confusing name

- Does not “break” the flow of the function
- A more apt synonym is available, `pure`

```
getUpper = getChar >=> \c -> return (toUpper c)
```

```
-- getChar >=> return . toUpper
```

Uppercase input

We need a way to *embed* pure computations, like `toUpper`, in the impure world

```
return :: a -> IO a
```

```
return a = \w -> (a, w)
```

Warning! `return` is indeed a very confusing name

- Does not “break” the flow of the function
- A more apt synonym is available, `pure`

```
getUpper = getChar >=> \c -> return (toUpper c)
```

```
-- getChar >=> return . toUpper
```

There is no **safe, general** function that runs an arbitrary IO action and returns its result as a pure value

```
backFromHell :: IO a -> a
```

Why?

There is no **safe, general** function that runs an arbitrary IO action and returns its result as a pure value

```
backFromHell :: IO a -> a
```

Why?

In this way we ensure that the outside world never “infects” pure expressions

- Referential transparency is preserved

I0 is abstract: never see World

Mixing IO and recursion

We cannot directly pattern match on the result inside an IO action

- We often use case expressions after (>>=)

```
mystery :: IO String
mystery = getChar >>= (\c ->
    case c of
        '\n' -> return []
        _      -> mystery >>= (\rest ->
            return (c : rest)
        )
    )
```

What does this code do?

Mixing IO and recursion

We cannot directly pattern match on the result inside an IO action

- We often use case expressions after (>>=)

```
mystery :: IO String
mystery = getChar >>= (\c ->
    case c of
        '\n' -> return []
        _      -> mystery >>= (\rest ->
            return (c : rest)
        )
    )
```

What does this code do?

Working directly with (>>=) is very cumbersome!

do-notation

Luckily, Haskell has do-notation for sequencing actions

```
getLine = do c <- getChar
            case c of
              '\n' -> return []
              _     -> do rest <- getLine
                        return (c : rest)
```

Blocks for IO start with the keyword do

- <- gives a name to the result of an IO action
- The notation was chosen to “look imperative”
- Unlike let, <- sequences an action and names its result

Cooking putStr

Let us write `putStr` with the new combinators

Cooking putStr

Let us write putStr with the new combinators

```
putStr :: String -> IO ()
```

```
putStr []      = return ()
```

```
putStr (c:cs) = putChar c >>= (\_ -> putStr cs)
```

Cooking putStr

Let us write putStr with the new combinators

```
putStr :: String -> IO ()
putStr []      = return ()
putStr (c:cs) = putChar c >>= (\_ -> putStr cs)
```

What is happening is much clearer with do-notation

```
putStr :: String -> IO ()
putStr []      = return ()
putStr (c:cs) = do putChar c
                   putStr cs
```

do-notation, in general

A general do block is translated as nested ($\gg=$)

```
do x1 <- a1          a1 >>= (\x1 ->
  x2 <- a2          a2 >>= (\x2 ->
  ...              ==>    ...
  xn <- an          an >>= (\xn ->
  expr              expr) ... ))
```

In addition, if you don't care about a value, you can write simply `ai` instead of `_ <- ai`

Rule of thumb: write with `do`; use its translation to reason about sequencing

Reuse an action or its result?

```
foo = do x <- getLine  
        return ()
```

```
bar = do let readAgain = getLine  
        x <- readAgain  
        y <- readAgain  
        return ()
```

```
baz = do let readAgain = getLine  
        let readAgain = getLine  
        x <- readAgain  
        return ()
```

Predict: how many lines does each read? Does inlining `readAgain` change that?

Guess a number

Pick a number between 1 and 100.

Is it 50? (g = greater, l = less, c = correct)

g

Is it 75? (g = greater, l = less, c = correct)

l

Is it 62? (g = greater, l = less, c = correct)

g

Is it 68? (g = greater, l = less, c = correct)

l

Is it 65? (g = greater, l = less, c = correct)

c

Guessed

Guess a number

Binary search: guess the midpoint of the remaining interval.

```
guess :: Int -> Int -> IO ()
```

```
guess l u
```

```
  | l > u = putStrLn "No number fits those answers."
```

```
  | otherwise = do
```

```
    let m = l + ((u - l) `div` 2)
```

```
    putStr ("Is it " ++ show m ++ "? ")
```

```
    putStrLn "(g = greater, l = less, c = correct)"
```

```
    k <- getLine
```

```
    case k of
```

```
      "g" -> guess (m + 1) u
```

```
      "l" -> guess l (m - 1)
```

```
      "c" -> putStrLn "Guessed"
```

```
      _   -> do putStrLn "Please type g/l/c!"
```

```
              guess l u
```

Guess a number, main program

When a Haskell executable starts, the runtime executes the `main` action

- We normally give `main` the type `IO ()`

```
import System.Environment (getArgs)
```

```
main :: IO ()
```

```
main = do (l:u:_) <- getArgs  
         guess (read l) (read u)
```

- `getArgs :: IO [String]` obtains program arguments
- `read :: Read a => String -> a`
 - Parses a `String` into a value
 - Here we assume at least two arguments containing valid `Int` values

Summary of basic I/O actions

`return` :: ???

`(>>=)` :: ???

`getChar` :: ???

`getLine` :: ???

`getArgs` :: ???

`putChar` :: ???

`putStr` :: ???

`putStrLn` :: ???

Summary of basic I/O actions

```
return    :: a -> IO a  
(>>=)    :: IO a -> (a -> IO b) -> IO b
```

```
getChar   :: IO Char  
getLine   :: IO String  
getArgs   :: IO [String]
```

```
putChar    :: Char -> IO ()  
putStr     :: String -> IO ()  
putStrLn   :: String -> IO ()
```

Dealing with files

The simplest functions to work with files in Haskell

```
type FilePath = String
```

```
readFile  :: ???
```

```
writeFile :: ???
```

Dealing with files

The simplest functions to work with files in Haskell

```
type FilePath = String
```

```
readFile  :: FilePath -> IO String
```

```
writeFile :: FilePath -> String -> IO ()
```

The following functions are often convenient

```
lines    :: String -> [String]  -- break at '\n'
```

```
unlines  :: [String] -> String  -- join lines
```

```
-- convert back and forth
```

```
show :: Show a => a -> String
```

```
read  :: Read a => String -> a
```

Guess a number, bounds from file

Assume the first two lines contain valid Int values.

```
main :: IO ()
main = do -- Read the contents of the file
          config <- readFile "guess.config"
          -- Get the first two lines
          let l:u:_ = lines config
          -- Parse the numbers and start guessing
          guess (read l) (read u)
```

I0 as first-class citizens

I/O actions are first-class

In the same way as you do with functions

- An I/O action can be an argument or result of a function
- I/O actions can be put in a list or other container

```
map (\name -> putStrLn ("Hello, " ++ name))  
  ["Mary", "John"]  ::  [IO ()]
```

Building versus execution of IO actions

```
map (\name -> putStrLn ("Hello, " ++ name))  
    ["Mary", "John"]  ::  [IO ()]
```

Evaluating this expression builds a list; it does **not** execute its actions

- We say that it *builds* the IO actions: describes what needs to be done but does not do it yet

To obtain the side-effects, you need to *execute* the actions

- At the interpreter prompt
- When sequenced into an action ultimately executed by `main`
- An executed action has type `IO a`

Execute a list of actions

`sequence_` as performs the side-effects of a list of actions

1. Define the type

```
sequence_ :: [IO a] -> IO ()
```

Execute a list of actions

`sequence_` as performs the side-effects of a list of actions

1. Define the type

```
sequence_ :: [IO a] -> IO ()
```

2. Enumerate the cases

```
sequence_ [] = _
```

```
sequence_ (a:as) = _
```

Execute a list of actions

`sequence_` as performs the side-effects of a list of actions

1. Define the type

```
sequence_ :: [IO a] -> IO ()
```

2. Enumerate the cases

```
sequence_ [] = _
```

```
sequence_ (a:as) = _
```

3. Define the cases

```
sequence_ [] = return ()
```

```
sequence_ (a:as) = do a  
                    sequence_ as
```

Execute a list of actions

We have all the ingredients to greet a list of people

```
greet :: [String] -> IO ()
```

```
greet = sequence_
```

```
    . map (\name -> putStrLn ("Hello, " ++ name))
```

Execute a list of actions

We have all the ingredients to greet a list of people

```
greet :: [String] -> IO ()  
greet = sequence_  
      . map (\name -> putStrLn ("Hello, " ++ name))
```

This combination is very common, so the library defines

```
mapM_ :: (a -> IO b) -> [a] -> IO ()  
  
greet = mapM_ (\name -> putStrLn ("Hello, " ++ name))
```

Execute a list of actions

By just flipping the order of arguments, we can write “imperative-looking” code

```
forM_ :: [a] -> (a -> IO b) -> IO ()
```

```
forM_ = flip mapM_
```

```
greet names = forM_ names $ \name ->
    putStrLn ("Hello, " ++ name)
```

Answering a yes-no question

`poseQuestion q` prints a question to the screen, obtains a y or n input from the user and returns it as a Boolean

```
poseQuestion :: String -> IO Bool
```

```
poseQuestion q
```

```
  = do putStr q
```

```
      putStrLn "Answer (y) or (n)"
```

```
      answer <- getLine
```

```
      case answer of
```

```
        ('y':_) -> return True
```

```
        ('n':_) -> return False
```

```
        _       -> do putStrLn "Answer (y) or (n)"
                      poseQuestion q
```

Gathering all answers

Once again, if we map over the list the actions are inside

```
map poseQuestion qs :: [IO Bool]
```

sequence_ does not work, since it throws away the result

```
sequence :: [IO a] -> IO [a]
```

...

Gathering all answers

Once again, if we map over the list the actions are inside

```
map poseQuestion qs :: [IO Bool]
```

sequence_ does not work, since it throws away the result

```
sequence :: [IO a] -> IO [a]
```

```
sequence [] = return []
```

```
sequence (a:as) = do r  <- a  
                   rs <- sequence as  
                   return (r:rs)
```

Reason about order: for a finite list, what changes if we swap the two <- lines?

Gathering all answers

Now we can gather answers to all questions at once

```
poseQuestions :: [String] -> IO [Bool]
poseQuestions = sequence . map poseQuestion
```

We have non-forgetful versions of the previous functions

```
mapM :: (a -> IO b) -> [a] -> IO [b]
forM :: [a] -> (a -> IO b) -> IO [b]
```

Naming convention: a function which ends in `_` throws away information

```
liftM2 :: (a -> b -> c)  
        -> IO a -> IO b -> IO c
```

```
liftM2 :: (a -> b -> c)
        -> IO a -> IO b -> IO c

liftM2 f ia ib = do
  a <- ia
  b <- ib
  return (f a b)
```

Randomness

Random generation

Random generation is provided by the `System.Random` module of the `random` package

```
class Random a where
```

```
  randomR :: RandomGen g => (a, a) -> g -> (a, g)
```

```
  random  :: RandomGen g =>                g -> (a, g)
```

- `a` is the type of value you want to generate
- `g` is the type of random generators
 - It carries a *state*, initially determined by a *seed*
 - These functions are pure: the same state gives the same result

Generating several random numbers

To generate successive values with `random`, thread the generator state

```
generateTwoNumbers :: RandomGen g  
                  => g -> ((Int, Int), g)  
generateTwoNumbers g  
    = ...
```

Generating several random numbers

To generate successive values with `random`, thread the generator state

```
generateTwoNumbers :: RandomGen g
                  => g -> ((Int, Int), g)

generateTwoNumbers g
  = let (v1, g1) = random g
        (v2, g2) = random g1 -- Use updated state
    in ((v1, v2), g2)        -- Return final state
```

Diagnose: what changes if the second call uses `g`? Must the correct version produce two different numbers?

Obtaining the seed

A fixed seed gives repeatable tests; external input can initialise a fresh generator

- We have RandomGen instance StdGen
- getStdRandom uses the current global generator and stores the updated generator afterwards

```
getStdRandom :: (StdGen -> (a, StdGen)) -> IO a
```

- Note the use of a higher-order function to encapsulate the part of the program which needs randomness

Because of their ubiquity, the following functions are provided

```
randomRIO = getStdRandom . randomR
```

```
randomIO  = getStdRandom    random
```

Summary

- Introduced purity/referential transparency and contrasted it with side effects

Summary

- Introduced purity/referential transparency and contrasted it with side effects
- Actions with side-effects which return a value of type `a` are represented by `IO a`
 - `IO` makes effectful actions explicit in types
 - `a -> IO b` builds an action from a pure input
 - The runtime executes `main`, conventionally of type `IO ()`

Summary

- Introduced purity/referential transparency and contrasted it with side effects
- Actions with side-effects which return a value of type `a` are represented by `IO a`
 - `IO` makes effectful actions explicit in types
 - `a -> IO b` builds an action from a pure input
 - The runtime executes `main`, conventionally of type `IO ()`
- To sequence `IO` actions, use `do`-notation
 - Under the hood it translates to nested `(>=>)` (`bind`)

Summary

- Introduced purity/referential transparency and contrasted it with side effects
- Actions with side-effects which return a value of type `a` are represented by `IO a`
 - `IO` makes effectful actions explicit in types
 - `a -> IO b` builds an action from a pure input
 - The runtime executes `main`, conventionally of type `IO ()`
- To sequence `IO` actions, use `do`-notation
 - Under the hood it translates to nested `(>>=)` (`bind`)
- `IO` actions are first-class citizens